
Atividade 1 (AV1) — Relatório Técnico

Recuperação de contexto para IA generativa em materiais educacionais abertos

Corretude, eficiência e recuperação de contexto:
um estudo comparativo de estratégias de ordenação e busca sobre um livro-texto aberto

João Cosme Sena Sá · Eduardo Henrique do Lago Silva · Helena Carvalho Leal
Hernandison da Silva Bispo · Nadianne Maria dos Santos Galvão · Rafael Takeguma Goto

Tema	8 — Materiais educacionais abertos
Corpus	<i>Pense Python</i> (tradução livre da 3ª ed. de <i>Think Python</i>)
Repositório	https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_N
Licença do código	MIT · Versão/tag: v0.1.0
Vídeo da atividade	<i>URL a preencher até 22/09/2026</i> (também em README, VIDEO.md e Classroom)

Estado desta versão do relatório. Este documento consolida a especificação formal, os algoritmos e pseudocódigos, a prova de corretude, a análise no modelo RAM, as recorrências, a metodologia experimental, os resultados obtidos e a relação com RAG.

As figuras teóricas são explicitamente identificadas como diagramas ou curvas derivadas das funções de complexidade, enquanto as tabelas e gráficos da Seção 12 apresentam resultados experimentais obtidos a partir da execução do pipeline.

O experimento principal foi concluído com 3600 execuções mensuráveis, cobrindo quatro configurações, três tamanhos de corpus, 30 consultas, dois valores de k e cinco repetições, além de duas execuções de aquecimento descartadas por cenário.

Sumário

1	Identificação da equipe	2
2	Tema, corpus e fonte	3
3	Repositório, licença, versão e data de acesso	4
4	Definição formal do problema	4
4.1	Entradas e saída	4
4.2	Função de relevância	4
4.3	Pré-condições, pós-condições e casos de borda	5
5	Representação dos dados e estratégia de chunking	5
5.1	Normalização	5
5.2	Segmentação (chunking)	5
5.3	Representações computacionais	6
6	Algoritmos e pseudocódigo	6
6.1	Busca linear com pontuação (comum a C1 e C3)	6
6.2	Insertion Sort — Config. 1	6
6.3	Merge Sort — Config. 3 e estratégia de divisão e conquista	6
6.4	Índice invertido e busca binária — Config. 2	7
6.5	Seleção dos k melhores (comum a todas)	7
7	Justificativa de corretude	8
7.1	Validação prática da ordenação	9
8	Análise no modelo RAM	9

8.1	Parâmetros	9
8.2	Operações elementares e custo por algoritmo	9
9	Melhor, pior e caso médio	10
10	Recorrências	11
10.1	Merge Sort	11
10.2	Busca binária	11
11	Metodologia experimental	12
11.1	Configurações comparadas	12
11.2	Cargas, repetições e execuções	12
11.3	Métricas registradas	12
11.4	Reprodutibilidade	13
11.5	Referência de relevância	13
12	Resultados e gráficos	13
12.1	Tempo por etapa e configuração	13
12.2	Qualidade da recuperação e memória	14
12.3	Escalabilidade medida	15
13	Discussão de escalabilidade	16
14	Relação com IA generativa e RAG	17
15	Limitações e ameaças à validade	17
16	Declaração de uso de IA generativa	17
17	Contribuição individual	18
18	Vídeo da atividade	18
19	Referências	19

1. Identificação da equipe

Equipe de seis discentes do curso de Ciência da Computação (PROCC/UFS), na disciplina Projeto e Análise de Algoritmos, período 2026.2. A modalidade é trabalho em equipe (5 a 7 integrantes).

Integrante	Frente principal
João Cosme Sena Sá	Coordenação e entrega; dados de teste e pacote; índice invertido e busca binária no vocabulário (Config. 2)
Eduardo Henrique do Lago Silva	Experimentos, tabelas, gráficos e baseline de referência scikit-learn (Config. 4)
Helena Carvalho Leal	Consultas, julgamentos de relevância (qrels), métricas (Precision@k), relação com IA e limitações
Hernandison da Silva Bispo	Ingestão, normalização, fragmentação e manutenção do relatório
Nadianne Maria dos Santos Galvão	Corretude, modelo RAM, complexidade e recorrências
Rafael Takeguma Goto	Busca linear, Insertion Sort, Merge Sort, pontuação e consulta (Config. 1 e 3)

A divisão acima é a **divisão de frentes** acordada (`docs/CONTRIBUICOES.md`); a revisão de código, da prova e dos resultados é responsabilidade coletiva. A tabela de contribuição individual **com evidências** é mantida em `docs/CONTRIBUICOES.md` e resumida na Seção 17.

2. Tema, corpus e fonte

O tema é o de número 8 do enunciado, **materiais educacionais abertos**. O corpus é o livro *Pense Python*, tradução livre da 3ª edição de *Think Python*, de Allen B. Downey, traduzida por Rodrigo Castelan Carlson — conteúdo didático público de introdução à programação em Python, sem qualquer dado sensível, pessoal ou sigiloso.

Campo	Registro
Nome	<i>Pense Python</i> — tradução livre da 3ª ed. de <i>Think Python</i>
Autor / Tradução	Allen B. Downey / Rodrigo Castelan Carlson
Fonte	https://rodrigocarlson.github.io/PensePython3ed/ ; https://github.com/rodrigocarlson/PensePython3ed
Commit de referência	cfde2c450bf4f2ea65326da88e9968f400597e92 (branch main)
Recorte	21 notebooks Jupyter: prefácio, introdução e capítulos 1 a 19 (<code>capitulos/*.ipynb</code>)
Tamanho auditado	817.578 bytes · 2.789 células · ≈73.867 palavras
SHA-256 do recorte	2010626258...5768dc5 (agregado; ver <code>data/corpus_manifest.csv</code>)
Idioma	Português brasileiro (com estrangeirismos técnicos do Python/Jupyter)
Licença do texto / código	CC BY-NC-SA 4.0 / MIT
Data de acesso	02/09/2026 (aquisição registrada); planejamento em 01/09/2026
Dados sensíveis	Nenhum — conteúdo didático público

Corpus não versionado, mas reproduzível. Como a licença do texto é não comercial e com compartilhamento pelas mesmas condições (CC BY-NC-SA), o texto não é redistribuído no repositório (cujo código é MIT). O script `scripts/baixar_corpus.py` baixa o material no commit fixado e `data/corpus_manifest.csv` guarda o SHA-256 de cada notebook e um **hash agregado do recorte**, permitindo conferir que qualquer download reproduz o mesmo conjunto auditado. Trechos citados preservam atribuição ao autor e ao tradutor.

Pergunta(s) de recuperação. As consultas seguem o formato “tira-dúvidas” de um estudante sobre o livro, por exemplo: “*como funciona um laço for em Python?*”, “*o que é uma função recursiva?*”, “*como tratar exceções com try e except?*”. O conjunto completo de 30 consultas está em `data/queries.csv`, redigido **antes** de qualquer resultado ser observado.

3. Repositório, licença, versão e data de acesso

Item	Registro
URL do repositório	https://github.com/joaocss/PAA_UFS_2026_2_SenaSa_Joao_Bispo_Hernandison_Galvao_Nadianne_Goto_Rafael_Leal_Helena_Silva_Eduardo
Licença do código	MIT (arquivo LICENSE)
Versão / tag	v0.1.0 (CITATION.cff, date-released: 2026-09-03)
Commit/tag/release do repositório	a fixar na entrega final (tag da release de 23/09/2026)
Data de acesso ao corpus	02/09/2026 (a reconfirmar no dia da aquisição)
Ambiente travado	requirements.lock: pytest 8.3.4, scikit-learn 1.6.1, matplotlib 3.10.0, numpy 2.2.2

O repositório não contém chaves, senhas, tokens, dados pessoais ou artefatos sem licença compatível. Arquivos grandes são controlados por `.gitignore`; as instruções de obtenção e reprodução estão no `README.md`.

4. Definição formal do problema

O protótipo é um **recuperador de contexto**: dada uma consulta, devolve os trechos mais relevantes do corpus, ordenados por relevância. Não é um chatbot; o interesse está nas operações algorítmicas anteriores à geração de texto.

4.1 Entradas e saída

- **Corpus** $C = \langle c_1, \dots, c_N \rangle$: sequência de N chunks. Cada chunk c_i tem um `chunk_id` textual único, um `source_order` id_i inteiro único em $0..N-1$ (usado no desempate), o texto normalizado e $origem_i = (\text{notebook}, \text{seção}, \text{offset})$.
- **Consulta** q : cadeia em português; após normalização vira um multiconjunto de m termos.
- $k \in \mathbb{N}$: número máximo de trechos a devolver.
- **Saída** R : sequência de até k chunks de C , sem repetição, ordenada de forma não crescente pela função de relevância.

4.2 Função de relevância

A pontuação é **TF-IDF** lexical, definida pela equipe e usada de forma idêntica nas três configurações obrigatórias:

$$\begin{aligned} \text{tf}(t, c) &= 1 + \ln f_{t,c} \quad (\text{se } f_{t,c} > 0; \text{ senão } 0), \\ \text{idf}(t) &= \ln \left(\frac{N+1}{\text{df}(t)+1} \right) + 1, \\ \text{score}(q, c) &= \sum_{t \in \text{Termos}(q)} q t f_t \cdot \text{tf}(t, c) \cdot \text{idf}(t). \end{aligned}$$

$q t f_t$ representa a frequência do termo t na consulta, $f_{t,c}$ é a frequência do termo t no chunk c e $\text{df}(t)$ é o número de chunks que contêm t . Termos ausentes do vocabulário não contribuem para a pontuação.

A suavização adotada para o IDF segue a mesma forma utilizada pela configuração de referência com scikit-learn, enquanto a ponderação TF-IDF da implementação própria foi mantida explícita para permitir a comparação entre as configurações.

A **ordem total** $\preceq$ é:

$$c_a \preceq c_b \iff \text{score}(q, c_a) > \text{score}(q, c_b) \vee (\text{scores iguais} \wedge id_a < id_b),$$

onde id denota o **source_order** do chunk. O desempate determinístico por **source_order** crescente garante que o resultado independe da ordem de entrada e da configuração usada.

4.3 Pré-condições, pós-condições e casos de borda

Categoria	Especificação
Pré-condições	$N \geq 0$; $k \geq 0$; todo score é real finito; os source_order são únicos; $\preceq$ é ordem total.
Pós-condições	Seja n' o número de chunks com score > 0 . Então $ R = \min(k, n')$; todo elemento de R pertence a C e é único; R está ordenado por $\preceq$; e não existe chunk fora de R com relevância maior (por $\preceq$) que qualquer elemento de R — ou seja, R são os $\min(k, n')$ melhores.
Casos de borda	consulta vazia $\rightarrow R = \langle \rangle$; $k = 0 \rightarrow R = \langle \rangle$; $k > n' \rightarrow$ devolve os n' com score > 0 ; todos os scores $0 \rightarrow R = \langle \rangle$; empates $\rightarrow$ resolvidos por source_order ; termo fora do vocabulário $\rightarrow$ ignorado.
Hipótese sobre os dados	Para a vantagem do índice invertido (Config. 2) supõe-se esparsidade : $df \ll N$ para os termos típicos. A corretude não depende dessa hipótese; apenas a eficiência.

5. Representação dos dados e estratégia de chunking

5.1 Normalização

Deliberadamente simples, para poder ser auditada:

- **Células de texto (Markdown)**: NFKC, minúsculas, separação controlada da pontuação e compactação de espaços; **acentos preservados**.
- **Células de código**: fonte intacta, com uma visão lexical à parte (tokens), sem alterar indentação.
- **Stopwords não são removidas** na configuração principal: em português isso apagaria termos funcionais que mudam o sentido de consultas como “*diferença entre while e for*”.
- Saídas de execução das células são descartadas.

5.2 Segmentação (chunking)

Respeita a hierarquia do livro — notebook, título e seção — e **nunca mistura** célula de texto com célula de código no mesmo chunk. A configuração principal usa **256 tokens** com **sobreposição de 32**, produzindo cerca de **2737 chunks**. As configurações 128/16 e 512/64 entram apenas no experimento de sensibilidade.

5.3 Representações computacionais

- **Saco de palavras / TF-IDF:** cada chunk é um vetor esparsos de pesos TF-IDF sobre V .
- **Índice invertido (Config. 2):** mapa termo $\rightarrow$ lista de *postings* $(id, f_{t,c})$, com o **vocabulário em vetor ordenado** para permitir **busca binária** por termo.

Riscos conhecidos. A tradução carrega termos em inglês do Python/Jupyter (ex.: *loop*, *string*), o que afeta consultas com e sem estrangeirismos; um único livro introdutório não representa todo o universo dos materiais educacionais abertos — os resultados valem como **estudo de caso**.

6. Algoritmos e pseudocódigo

Os algoritmos obrigatórios são cobertos assim: **busca linear** na pontuação de todos os chunks; **ordenação** por Insertion Sort e Merge Sort da equipe; **busca binária** no vocabulário do índice invertido; **divisão e conquista** pelo próprio Merge Sort; e **comparação com biblioteca** pelo TF-IDF do scikit-learn, usado apenas como referência (Config. 4).

6.1 Busca linear com pontuação (comum a C1 e C3)

```
funcao PontuarLinear(q, C, idf):  
  candidatos <- lista vazia  
  para cada chunk c em C faca # N iteracoes  
    s <- 0  
    para cada termo t em q faca # m iteracoes  
      se t in c.termos entao # teste O(1) via hash  
        s <- s + tf(t, c) * idf[t]  
      se s > 0 entao candidatos.anexar( (s, c.id, c) )  
  retorne candidatos
```

6.2 Insertion Sort — Config. 1

```
funcao InsertionSort(A, precede):  
  para i <- 1 ate |A|-1 faca  
    chave <- A[i]; j <- i - 1  
    enquanto j >= 0 e A[j] sucede chave faca # desloca os maiores  
      A[j+1] <- A[j]; j <- j - 1  
    A[j+1] <- chave  
  retorne A
```

6.3 Merge Sort — Config. 3 e estratégia de divisão e conquista

```
funcao MergeSort(A, precede):  
  se |A| <= 1 entao retorne A # caso base  
  meio <- floor(|A| / 2)  
  E <- MergeSort(A[0 .. meio-1]) # T(N/2)  
  D <- MergeSort(A[meio .. |A|-1]) # T(N/2)
```

```

retorne Merge(E, D) # Theta(N)

funcao Merge(E, D):
  0 <- lista vazia; i <- 0; j <- 0
  enquanto i < |E| e j < |D| faca
    se E[i] precede-ou-igual D[j] entao # estavel: empate -> E
      0.anexar(E[i]); i <- i + 1
    senao
      0.anexar(D[j]); j <- j + 1
  0.anexar(restante de E a partir de i)
  0.anexar(restante de D a partir de j)
  retorne 0

```

6.4 Índice invertido e busca binária — Config. 2

```

funcao BuscaBinaria(vocab, t): # vocab = vetor ORDENADO de termos
  lo <- 0; hi <- |vocab| - 1
  enquanto lo <= hi faca
    mid <- floor((lo + hi) / 2)
    se vocab[mid] = t entao retorne mid
    senao se vocab[mid] < t entao lo <- mid + 1
    senao hi <- mid - 1
  retorne NAO_ENCONTRADO

funcao BuscarIndexado(q, indice, estatisticas):
  termos <- termos distintos normalizados de q
  posicoes <- lista vazia

  para cada termo t em termos faca
    p <- BuscaBinaria(indice.vocabulario, t) # Theta(log V)
    se p != NAO_ENCONTRADO entao
      posicoes <- UnirPostings(
        posicoes,
        indice.postings[p]
      )

  candidatos <- lista vazia

  para cada posicao em posicoes faca
    c <- indice.chunks[posicao]
    s <- RelevanceScore(q, c, estatisticas)

    se s > 0 entao
      candidatos.anexar((c, s))

  retorne MergeSort(candidatos) # ordena apenas os candidatos

```

6.5 Seleção dos k melhores (comum a todas)

```

funcao TopK(resultados_ordenados, k):
  se k <= 0 entao
    retorne lista vazia

  candidatos <- lista vazia

```

```

para cada (chunk, score) em resultados_ordenados faca
  se score > 0 entao
    candidatos.anexar((chunk, score))

retorne os primeiros min(k, |candidatos|) elementos

```

Como a lista já está ordenada pela relação $\preceq$, a filtragem preserva essa ordem. Assim, os primeiros $\min(k, |candidatos|)$ elementos correspondem aos k resultados válidos mais relevantes.

7. Justificativa de corretude

Provamos a corretude do **Merge Sort** (Config. 3 e núcleo da Config. 2), por indução forte no tamanho da entrada, apoiada em um invariante de laço para o **Merge**. O texto completo está em docs/CORRETUDE.md.

Teorema. Dada uma lista finita A de chunks com identificadores únicos e a ordem total $\preceq$ (score decrescente; empate por id crescente), $\text{MergeSort}(A, \preceq)$ **termina** e devolve uma **permutação de A ordenada por $\preceq$** .

Hipóteses.

- A é finita; cada elemento tem score real finito e id único.
- $\preceq$ é ordem total: reflexiva, antissimétrica (pelo desempate por id único) e transitiva.
- A comparação de dois elementos custa $\Theta(1)$.

Lema (corretude do Merge) — invariante de laço.

Invariante. No início de cada iteração do laço **enquanto** de $\text{Merge}(E, D)$, com E e D já ordenados por $\preceq$: (i) a saída O está ordenada por $\preceq$; (ii) O contém exatamente os elementos já consumidos de E (índices $< i$) e de D (índices $< j$); e (iii) todo elemento de O é $\preceq$ a qualquer elemento ainda não consumido de E ou D .

- **Inicialização.** $O = \langle \rangle$ e $i = j = 0$; (i)–(iii) valem vacuamente.
- **Manutenção.** O algoritmo escolhe o menor entre $E[i]$ e $D[j]$ (empate a favor de E). Como E e D estão ordenados, o escolhido x é $\preceq$ a todos os não consumidos; pode ser anexado ao fim de O preservando (i) e (iii). O contador avança, mantendo (ii).
- **Término.** Cada iteração consome um elemento; o laço executa no máximo $|E| + |D|$ vezes. Os elementos restantes da lista não esgotada já são, por (iii), $\preceq$ a nada em O e estão em ordem; anexá-los mantém O ordenado. Ao final, O é uma permutação ordenada de $E \cup D$.

Indução no tamanho de A .

- **Base** ($|A| \leq 1$). Devolvida como está: trivialmente ordenada e permutação de si mesma.
- **Hipótese de indução.** MergeSort é correto para toda entrada de tamanho $< n$.
- **Passo** ($|A| = n > 1$). A é dividida em E ($\lfloor n/2 \rfloor$) e D ($n - \lfloor n/2 \rfloor$), ambas de tamanho $< n$. Por hipótese de indução, as chamadas recursivas devolvem permutações ordenadas de E e de D . Pelo Lema, $\text{Merge}(E, D)$ devolve uma permutação ordenada de $E \cup D$, que é A a menos de reordenação. ■

Término da recursão. Os tamanhos decrescem estritamente ($\lfloor n/2 \rfloor < n$ e $n - \lfloor n/2 \rfloor < n$ para $n > 1$) e atingem o caso base; logo a recursão é finita.

Estabilidade e desempate. O Merge anexa de E em caso de empate, tornando a ordenação **estável**. Como os *id* são únicos e entram em $\preceq$, não há empates verdadeiros entre chunks distintos: a saída é **determinística**, qualquer que seja a ordem de entrada ou a configuração (C1, C2 ou C3).

Corolário (top- k). Se A está ordenada por $\preceq$, seus primeiros $\min(k, |A|)$ elementos são exatamente os k maiores segundo $\preceq$. Logo TopK aplicado à saída do MergeSort devolve os k chunks mais relevantes, cumprindo a pós-condição da Seção 4.

Limites do argumento e casos em que não se aplica.

- Garante **ordenação correta e término**, não que o TF-IDF corresponda à intenção de quem consultou: relevância lexical é um *proxy* imperfeito (Seções 14 e 15).
- Se algum score for indefinido (NaN) ou o comparador não for transitivo, $\preceq$ deixa de ser ordem total e o teorema não se aplica.
- Se os *id* não forem únicos, o desempate falha e a saída pode deixar de ser determinística.
- A prova não trata de precisão de ponto flutuante; scores muito próximos podem ser tratados como empate e decididos pelo **source_order** — comportamento coberto pelos testes.

7.1 Validação prática da ordenação

Como complemento à prova, a relação de ordem $\preceq$ usada em todas as configurações é validada diretamente em `tests/unit/test_sort_key.py`, sobre a função `sort_key` de `src/paa_context/sort_key.py`, que estabelece maior **score** primeiro e, em empate, menor **source_order**. Três testes cobrem os pilares do argumento:

- `test_maior_score_vem_primeiro`: prioridade do maior **score**;
- `test_empate_usa_menor_source_order`: critério de desempate por **source_order**;
- `test_ordem_transitiva`: transitividade da relação (condição para $\preceq$ ser ordem total).

Os três passam isoladamente e a suíte completa do projeto é executada e aprovada (118 testes aprovados na suíte completa do projeto, incluindo os três testes específicos de `sort_key`). Esta subseção consolida a análise de corretude conduzida por Nadianne Galvão (`docs/CORRETUDE.md` e o documento de corretude, RAM, complexidade e recorrências).

8. Análise no modelo RAM

8.1 Parâmetros

8.2 Operações elementares e custo por algoritmo

Contam-se como operações elementares: comparação de chaves, acesso a vetor/lista, atribuição, incremento, chamada recursiva, consulta a tabela de dispersão e atualização de estrutura.

Insertion Sort (sobre C_q candidatos). O laço externo executa $C_q - 1$ vezes; o interno desloca, no pior caso, todos os já ordenados. Comparações $\sum_{i=1}^{C_q-1} i = C_q(C_q - 1)/2$ no pior caso, com igual

Símbolo	Significado
N	número de chunks (2737 na configuração principal)
V	tamanho do vocabulário (termos distintos)
L	total de tokens do corpus
m	número de termos da consulta
ℓ	número de termos do chunk analisado
k	número de resultados retornados
C_q	número de chunks candidatos (score > 0) para a consulta q
P	tamanho total das listas de postings percorridas = $\sum_{t \in q} \text{df}(t)$
r	número de repetições experimentais

ordem de acessos e atribuições. Como a implementação cria uma cópia da lista de entrada antes da ordenação, o espaço adicional é $O(C_q)$.

Merge Sort (sobre C_q candidatos). Cada nível percorre todos os elementos uma vez ($\Theta(C_q)$ por nível) e há $\lceil \log_2 C_q \rceil$ níveis, dando $\Theta(C_q \log C_q)$ comparações. Espaço $\Theta(C_q)$ pelas listas auxiliares.

Custo do pipeline por configuração.

Etapa	C1 (lin.+Insertion)	C2 (índice+bin.+Merge)	C3 (lin.+Merge)
Pontuação / candidatos	$\Theta(N(m + \ell))$	$O(m \log V + mP + C_q(m + \ell))$	$\Theta(N(m + \ell))$
Seleção top- k	$\Theta(C_q)$	$\Theta(C_q)$	$\Theta(C_q)$
Espaço adicional (consulta)	$\Theta(C_q)$	$\Theta(V + P_{\text{tot}}) + \Theta(C_q)$	$\Theta(C_q)$

Fatores fora do modelo RAM. O modelo ignora hierarquia de cache, custo de E/S ao ler os notebooks, alocação de memória, coletor de lixo do Python e a diferença entre código interpretado (C1–C3) e rotinas vetorizadas em C/NumPy (Config. 4). Por isso a Seção 12 mede tempo de parede além de contar operações.

9. Melhor, pior e caso médio

Algoritmo	Melhor	Pior	Médio	Espaço
Busca/pontuação linear	$\Theta(N(m + \ell))$	$\Theta(N(m + \ell))$	$\Theta(N(m + \ell))$	$\Theta(C_q)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(n)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Busca binária (por termo)	$\Theta(1)$	$\Theta(\log V)$	$\Theta(\log V)$	$\Theta(1)$
Consulta via índice (C2)	$O(m \log V + mP + C_q(m + \ell) + C_q \log C_q)$	$O(m \log V + mP + C_q(m + \ell) + C_q \log C_q)$	$O(m \log V + mP + C_q(m + \ell) + C_q \log C_q)$	$\Theta(V + P_{\text{tot}} + C_q)$

Onde $n = C_q$ é o número de candidatos a ordenar e ℓ é o número de termos do chunk analisado.

- **Insertion Sort** só é competitivo com C_q pequeno ou lista quase ordenada; melhor caso linear quando não há deslocamento.
- **Merge Sort** tem custo $\Theta(n \log n)$ **garantido** nos três casos — escolha segura para a comparação.

- **Na Configuração 2**, o custo depende fortemente da frequência dos termos consultados. Quando as listas de postings são pequenas, tanto P quanto C_q permanecem reduzidos, diminuindo o trabalho de união, pontuação e ordenação dos candidatos. No pior caso, termos muito frequentes podem fazer C_q se aproximar de N , reduzindo a vantagem do índice em relação à busca linear.

10. Recorrências

10.1 Merge Sort

$$T(N) = 2T(N/2) + \Theta(N), \quad T(1) = \Theta(1).$$

Origem dos termos: o fator **2** vem das duas chamadas recursivas (metades E e D); cada $T(N/2)$ ordena uma metade; o termo $\Theta(N)$ é o **Merge**, que percorre linearmente as duas metades. Pela **árvore de recorrência** (Figura 1), cada um dos $\lceil \log_2 N \rceil + 1$ níveis realiza trabalho $\Theta(N)$, somando $\Theta(N \log N)$. Pelo **Teorema Mestre**, com $a = 2$, $b = 2$, tem-se $N^{\log_b a} = N^1$ e $f(N) = \Theta(N)$: caso 2, dando $T(N) = \Theta(N \log N)$.

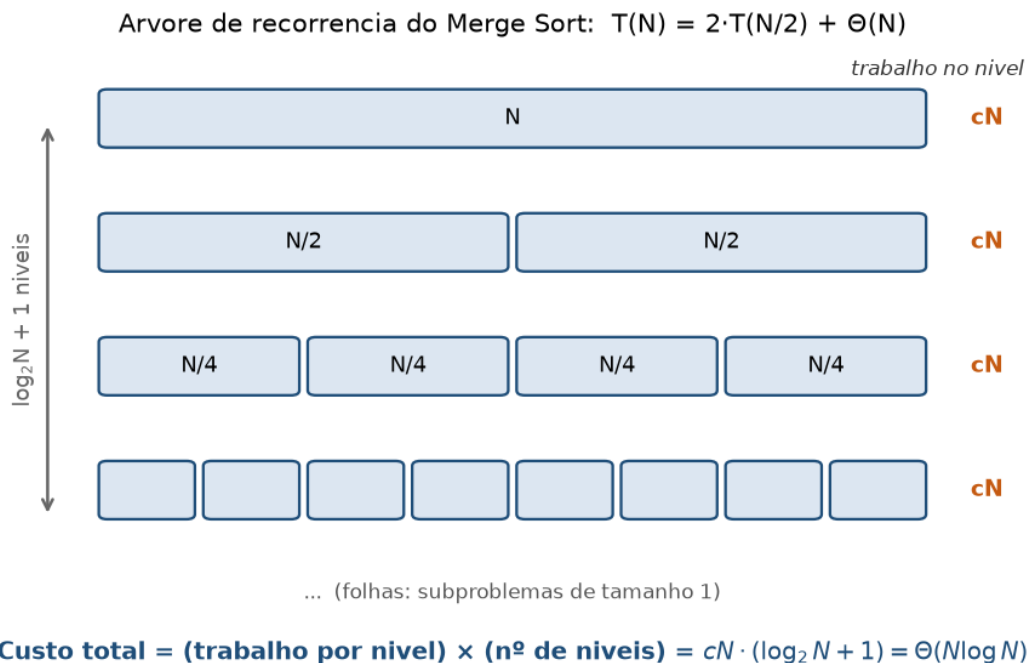


Figura 1: Árvore de recorrência de $T(N) = 2T(N/2) + \Theta(N)$: trabalho $\Theta(N)$ por nível e $\log_2 N + 1$ níveis, resultando em $\Theta(N \log N)$. Diagrama teórico.

10.2 Busca binária

$$T(V) = T(V/2) + \Theta(1), \quad T(1) = \Theta(1).$$

Origem dos termos: embora a implementação da busca binária seja iterativa, a recorrência $T(V) = T(V/2) + \Theta(1)$ modela a redução do intervalo de busca a cada iteração. Em cada passo, aproximadamente metade do vocabulário é descartada, enquanto o trabalho para calcular o ponto médio, comparar com `vocab[mid]` e atualizar os limites é constante, representado por $\Theta(1)$. Assim, após aproximadamente $\log_2 V$ reduções, o intervalo atinge tamanho unitário, resultando em $T(V) =$

$\Theta(\log V)$. A mesma recorrência também pode ser resolvida pelo Teorema Mestre, com $a = 1$ e $b = 2$, obtendo-se igualmente $T(V) = \Theta(\log V)$.

11. Metodologia experimental

11.1 Configurações comparadas

Config.	Descrição	Papel
C1 baseline	Varredura linear + Insertion Sort (equipe)	Baseline sem estrutura
C2 indexada	Índice invertido + busca binária + Merge Sort dos candidatos	Busca indexada/otimizada
C3 ordenada	Varredura linear + Merge Sort (equipe)	Divisão e conquista na ordenação
C4 referência	TF-IDF do scikit-learn	Somente baseline externo

As três obrigatórias usam **os mesmos dados e a mesma função de pontuação**; diferem só na estrutura de busca e na ordenação. Consequência para a validação: **C1, C2 e C3 devem produzir listas ranqueadas idênticas** (mesma ordem $\preceq$), diferindo apenas em custo — um teste de consistência forte. A Config. 4 pode divergir por usar normalização L2 e TF sublinear próprias.

11.2 Cargas, repetições e execuções

O experimento principal compreende **4 configurações $\times$ 3 tamanhos de corpus $\times$ 30 consultas $\times$ 2 valores de k $\times$ 5 repetições = 3600 execuções mensuráveis**. Antes das repetições medidas, são realizadas duas execuções de aquecimento por cenário, que são descartadas.

Os três tamanhos de corpus correspondem a aproximadamente 25% (684 chunks), 50% (1368 chunks) e 100% (2737 chunks), obtidos por subamostragem determinística por capítulo com semente fixa. Cada consulta é executada para $k = 5$ e $k = 10$.

Ao final das 3600 execuções, não foram observadas divergências entre os rankings produzidos por C1, C2 e C3. Esse resultado reforça que as três configurações implementadas preservam a mesma saída de recuperação sob a mesma função de relevância e o mesmo critério de ordenação, diferindo principalmente no custo computacional.

11.3 Métricas registradas

- Tempo de ingestão; de ordenação/construção de índice; de consulta; total.
- Memória aproximada (via `tracemalloc` e RSS).
- **Número de comparações**, por contador instrumentado.
- Tamanho do corpus, número de chunks e k .
- Precision@ k contra a referência manual de relevância construída para as 30 consultas.
- Taxa de falhas, resultados vazios ou irrelevantes; custo e limitações.

11.4 Reprodutibilidade

Todo o pipeline roda com `python scripts/reproduzir_tudo.py` (download → normalização → chunking → experimentos → figuras). Os scripts são **idempotentes**. Cada execução registra em `experimentos/logs`: hostname, processador, memória, SO, versão do Python, versões das bibliotecas, commit e a linha de comando completa. As consultas e a referência manual de relevância foram definidas antes da análise dos resultados, reduzindo o risco de viés de seleção.

11.5 Referência de relevância

A avaliação da qualidade utiliza uma referência de relevância construída manualmente para as 30 consultas. Para cada consulta foi definido um conjunto de chunks considerados pertinentes, acompanhado de justificativa. No cálculo de $\text{Precision}@k$, os chunks presentes nessa referência são tratados como relevantes e os demais como não relevantes.

12. Resultados e gráficos

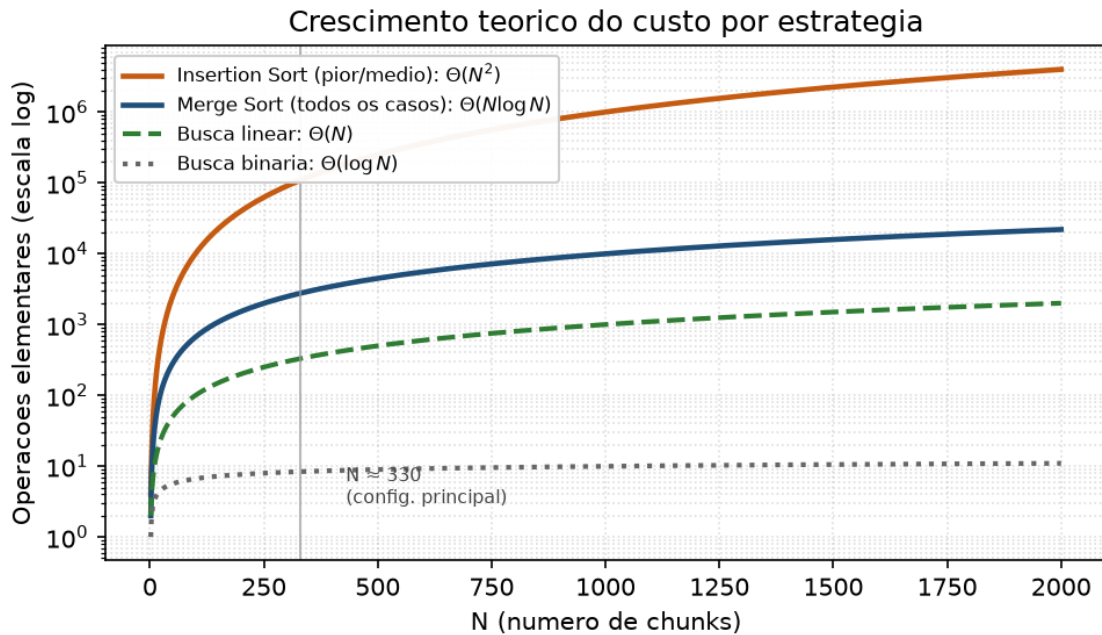


Figura 2: Crescimento teórico do número de operações elementares em função de N (escala log em y). A separação entre $\Theta(N^2)$ (Insertion), $\Theta(N \log N)$ (Merge) e $\Theta(\log N)$ (busca binária) motiva as três configurações. Curvas teóricas, não medições.

12.1 Tempo por etapa e configuração

Os resultados mostram que C1 apresentou o maior tempo total entre as três implementações próprias, associado principalmente ao custo da ordenação por Insertion Sort. C2 reduziu o tempo de consulta em relação a C1, embora tenha incorporado o custo de construção do índice. C3 apresentou o menor tempo total entre C1–C3 no corpus completo. A configuração C4 apresentou o menor tempo total mediano, sendo utilizada apenas como referência externa baseada em biblioteca.

Config.	N	Ingestão (ms)	Ord./índice (ms)	Consulta (ms)	Total (ms)	Comparações
C1	2737	39,31	63,87	128,03	166,81	461702
C2	2737	39,35	40,96	47,14	127,98	13039
C3	2737	39,47	2,34	65,55	105,95	12746,5
C4 (ref.)	2737	39,38	47,22	0,39	87,25	n/a

Tabela 1: Tempos medianos por etapa e contagem de comparações para o corpus completo.

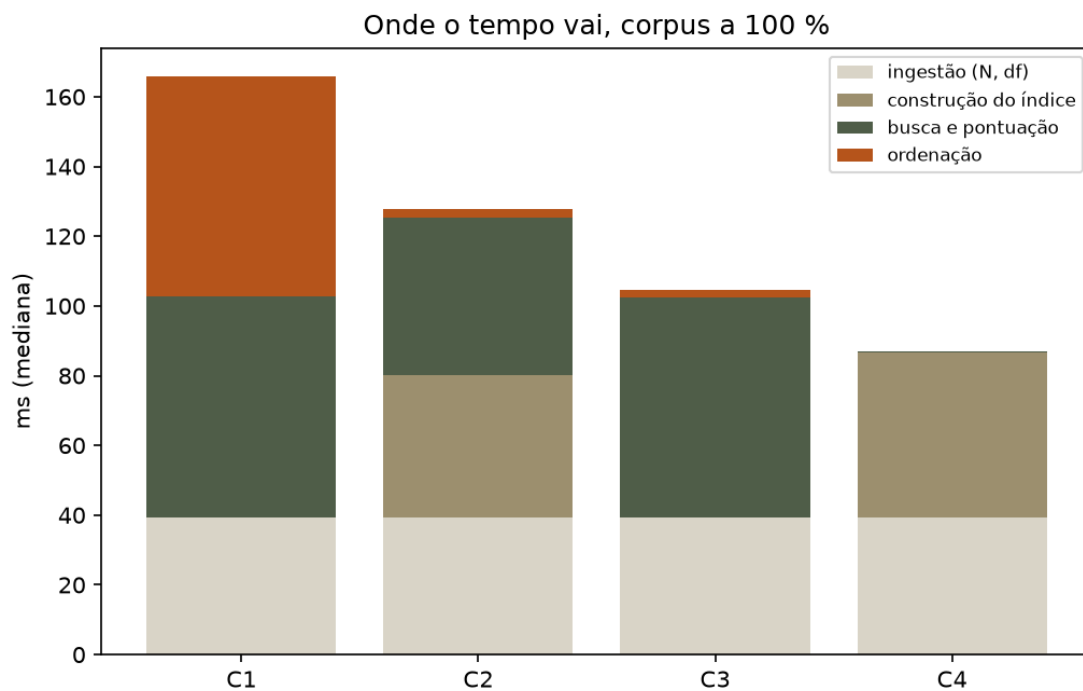


Figura 3: Decomposição do tempo mediano por etapa para o corpus completo ($N = 2737$). Em C1, a ordenação representa parcela relevante do custo; C2 incorpora o custo de construção do índice; C3 reduz fortemente o custo de ordenação; e C4 é apresentada apenas como referência externa.

12.2 Qualidade da recuperação e memória

Config.	Precision@5	Precision@10	Taxa de vazios	Memória pico (MB)
C1	0,0867	0,0767	0%	0,44
C2	0,0867	0,0767	0%	1,62
C3	0,0867	0,0767	0%	0,44
C4 (ref.)	0,2467	0,1833	0%	2,52

Tabela 2: Qualidade da recuperação e memória para o corpus completo ($N = 2737$).

12.3 Escalabilidade medida

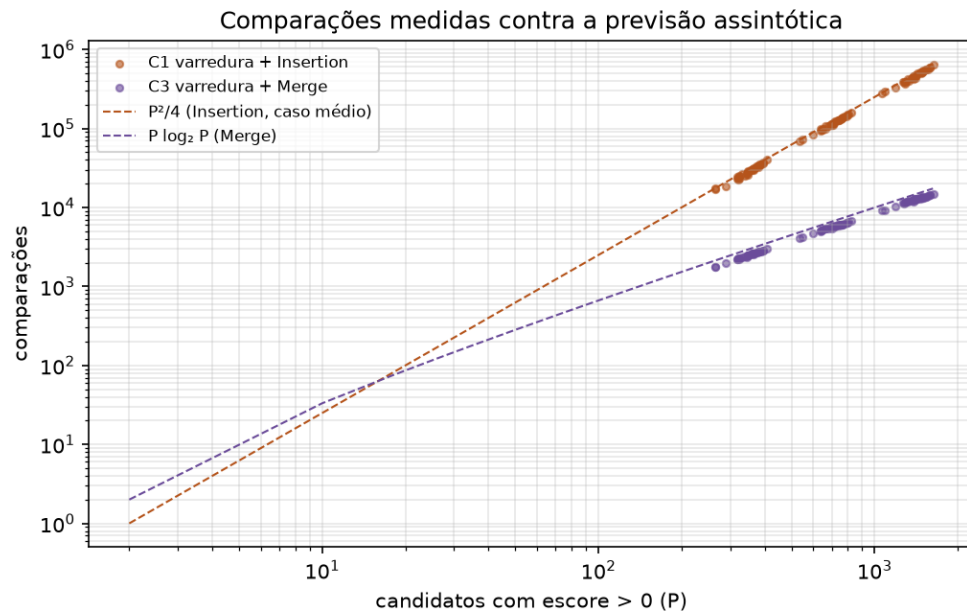


Figura 4: Número de comparações em função do número de candidatos com escore positivo. Os resultados de C1 apresentam crescimento compatível com o comportamento quadrático do Insertion Sort, enquanto C3 acompanha o crescimento $P \log_2 P$ esperado do Merge Sort.

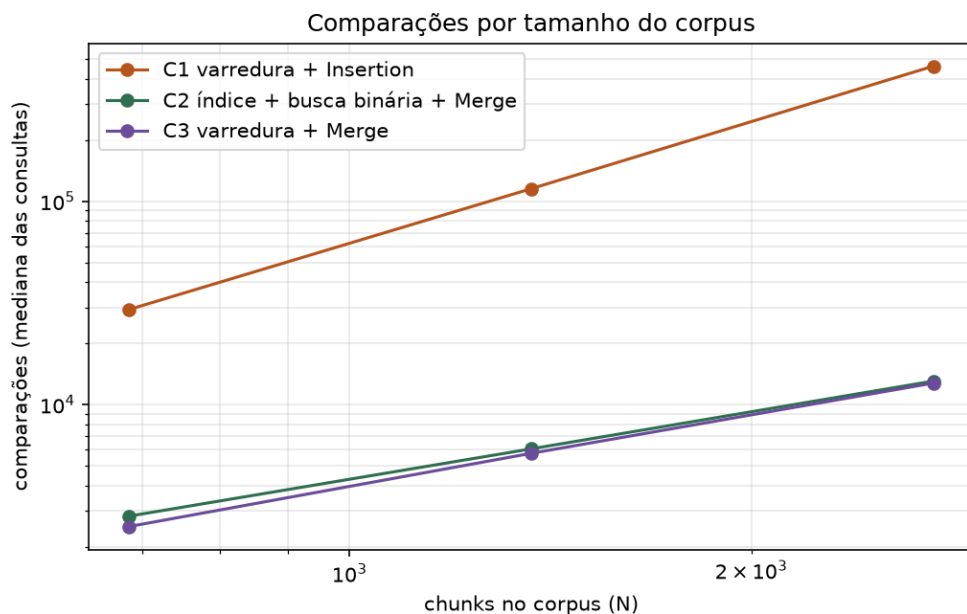


Figura 5: Mediana do número de comparações por tamanho do corpus. C1 apresenta crescimento mais acentuado, enquanto C2 e C3 permanecem em uma faixa significativamente menor de comparações.

N (chunks)	C1 total (ms)	C2 total (ms)	C3 total (ms)
684 (25%)	28,80	31,28	25,38
1368 (50%)	68,04	65,38	53,31
2737 (100%)	167,04	128,63	105,73

Tabela 3: Tempo total mediano por tamanho do corpus para as três configurações implementadas.

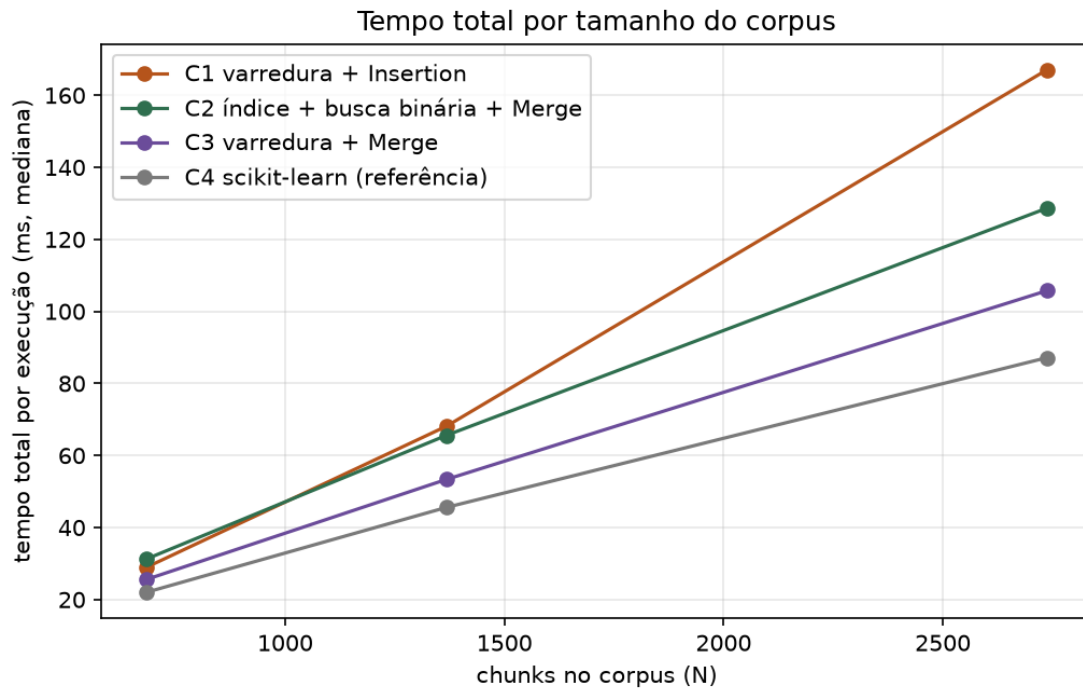


Figura 6: Tempo total mediano por execução em função do tamanho do corpus. O tempo cresce nas três configurações, com crescimento mais acentuado em C1. No corpus completo, C3 apresenta o menor tempo total entre as implementações próprias.

13. Discussão de escalabilidade

Para o corpus atual (~ 2737 chunks) as três configurações rodam em milissegundos; interessa a **tendência**. À medida que o acervo cresce (dezenas de livros, N na casa dos milhares):

- **C1** torna-se inviável: Insertion Sort cresce como C_q^2 e a varredura toca todos os N chunks por consulta.
- **C2** desacopla o custo de consulta do tamanho total: depende da **seletividade dos termos** (P , C_q) e de $\log V$, não de N . Pode apresentar vantagem crescente em acervos maiores quando os termos consultados são seletivos, ao custo de memória adicional para o índice.
- **C3** mantém a ordenação em $\Theta(C_q \log C_q)$, mas ainda paga a varredura $\Theta(N(m+\ell))$ por consulta.

Esse é o trade-off central da recuperação: gastar memória e tempo de indexação uma vez para reduzir o conjunto de chunks examinados em consultas seletivas, diminuindo o custo de recuperação em relação à varredura completa quando P e C_q permanecem pequenos. Bibliotecas como FAISS levam a ideia ao caso vetorial denso; aqui a demonstramos no caso lexical, com estruturas clássicas.

14. Relação com IA generativa e RAG

O contexto recuperado é a etapa de *retrieval* de um sistema RAG (*Retrieval-Augmented Generation*). Os sete pontos pedidos:

1. **Incorporação ao prompt.** Os k chunks são concatenados num bloco “Contexto: $\langle \text{chunks} \rangle \dots$ Pergunta: $\langle q \rangle$ ”, cada trecho com sua origem (notebook, seção) para citação.
2. **Impacto de k , tamanho de chunk e ordenação.** Mais/maiores chunks gastam mais tokens da janela, elevando latência e custo. A ordem importa (*lost in the middle*): pôr os mais relevantes primeiro, como a ordem $\preceq$ garante, é vantajoso.
3. **Lexical vs. semântico.** O TF-IDF capta **sobreposição de termos**, não sinônimos. Uma consulta por “*laço*” pode não casar com um trecho que diz “*loop*” — caso em que embeddings recuperariam e a busca lexical não.
4. **Riscos de recuperação ruim.** Incompleta, enviesada, desatualizada (commit fixo — desatualização controlada, mas real) ou irrelevante comprometem a resposta final.
5. **Afirmações não sustentadas.** O LLM pode “alucinar” além do recuperado; mitiga-se com instrução de responder só pelo contexto e citar a origem.
6. **Rastreabilidade.** Cada chunk carrega a proveniência, então a resposta cita a fonte exata — melhoria direta sobre um LLM sem recuperação.
7. **Trade-offs.** Tensão entre precisão, tempo, memória e custo de geração: k maior eleva o *recall* potencial, mas também o custo em tokens e o ruído; indexar acelera a consulta ao preço de memória.

15. Limitações e ameaças à validade

- **Construção.** A $\text{Precision}@k$ depende de uma referência manual de relevância, que envolve julgamento humano e pode conter subjetividade. Para reduzir esse risco, cada associação entre consulta e chunk relevante foi acompanhada de justificativa, mas a referência ainda é limitada ao conjunto de 30 consultas adotado no estudo.
- **Interna.** Medições de tempo em Python sofrem com coletor de lixo, cache e aquecimento; reportamos medianas, descartamos as duas primeiras execuções e contamos operações como métrica independente.
- **Externa.** Um único livro introdutório em português (domínio programação) não generaliza; vale como estudo de caso.
- **Viés do corpus.** A mistura de português e termos em inglês penaliza a busca lexical.
- **Conclusão.** Todas as medidas vêm de um único hardware; diferenças pequenas devem ser lidas junto da análise assintótica.

16. Declaração de uso de IA generativa

A declaração detalhada é mantida **durante** o trabalho em `docs/USO_DE_IA.md`, com ferramenta, modelo, finalidade, até cinco prompts reais, sugestões aproveitadas, sugestões corrigidas ou rejeitadas, erros identificados e a verificação aplicada. Resumo até esta versão:

Ferramenta / modelo	Finalidade	Verificação exigida
Assistente de IA generativa (registrar o modelo exibido na interface)	Apoio à estruturação e redação deste relatório e à formatação da prova e dos pseudocódigos.	Toda saída de IA foi conferida quanto à corretude, complexidade, casos de borda, estabilidade de ordenação, desempate e coerência entre análise e experimento. Código, prova ou análise não verificados não foram incorporados ao trabalho.

Compromisso de uso crítico. A prova da Seção 7 foi validada por testes de ordenação e desempate; os pseudocódigos foram confrontados com a implementação e os testes unitários; e as análises da Seção 12 foram verificadas por meio das execuções experimentais.

17. Contribuição individual

O enunciado pede contribuição **com evidência**, não cargo. A tabela final, com commits, arquivos e execuções, é mantida em `docs/CONTRIBUICOES.md`. Todos participam da revisão coletiva e da apresentação e do vídeo.

Integrante	Frente	Evidências (a consolidar)
João Cosme Sena Sá	Índice invertido / busca binária (C2); dados de teste, pacote e protocolo	<code>indice.py</code> , <code>busca_binaria.py</code> , <code>modelos.py</code> , <code>fixtures</code>
Eduardo Henrique do Lago Silva	Experimentos / figuras / baseline sklearn (C4)	<code>rodar_experimentos.py</code> , <code>gerar_figuras.py</code> , análise de C4
Helena Carvalho Leal	Consultas / qrels / métricas / IA / limitações	<code>queries.csv</code> , <code>qrels.csv</code> , <code>metricas.py</code>
Hernandison da Silva Bispo	Ingestão / normalização / fragmentação / relatório	<code>preprocessing.py</code> , <code>fragmentacao.py</code> , <code>scripts/</code> , <code>relatorio/</code>
Nadianne Maria dos Santos Galvão	Corretude / RAM / complexidade / recorrências	<code>CORRETUDE.md</code> , <code>test_sort_key.py</code> , Seções 7–10
Rafael Takeguma Goto	Busca linear / Insertion / Merge / pontuação (C1 e 3)	<code>linear_search.py</code> , <code>insertion_sort.py</code> , <code>merge_sort.py</code> , <code>relevance.py</code>

18. Vídeo da atividade

URL: a preencher até 22/09/2026. O vídeo terá até 10 minutos, com participação de todos e identificação clara de cada contribuição. A mesma URL constará no `README.md` (seção “Vídeo da atividade”), na capa deste relatório, no `VIDEO.md` (com data de gravação e participantes) e no Google Classroom quando solicitado. O vídeo apresentará: identificação da equipe, tema e corpus; o problema de recuperação; os algoritmos; uma demonstração breve; a corretude; a análise assintótica e as recorrências; os resultados e o gráfico principal; a relação com RAG; e as limitações e decisões de projeto.

19. Referências

1. CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.
2. KLEINBERG, J.; TARDOS, É. *Algorithm Design*. Pearson, 2006.
3. SKIENA, S. S. *The Algorithm Design Manual*. 3. ed. Springer, 2020.
4. SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.
5. MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
6. LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020. arXiv:2005.11401.
7. ZHOU, Z. et al. A Survey on Efficient Inference for Large Language Models. arXiv:2404.14294, 2024.
8. GAN, A. et al. Retrieval Augmented Generation Evaluation in the Era of LLMs: A Comprehensive Survey. arXiv:2504.14891, 2025.
9. FAISS. *Faiss documentation*. <https://faiss.ai/>.
10. SENTENCE TRANSFORMERS. <https://github.com/huggingface/sentence-transformers>.
11. PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *JMLR*, v. 12, 2011.
12. DOWNEY, A. B. *Think Python*. 3. ed. O'Reilly, 2024. Tradução *Pense Python* por CARLSON, R. C. <https://rodrigocarlson.github.io/PensePython3ed/>.